



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

TERMINSKI PLAN POIZVEDBE PROJEKTA BOOKBOX

BookBox – Anže Jeger, Lan Podlesnik, Nik Deželak

Maribor, april 2026

1 Uvod

V okviru projektnih predmetov bomo razvili sistem **BookBox**, katerega namen je omogočiti enostavnejšo izposajo in vračilo knjig s pomočjo pametnega paketnika. Sistem bo sestavljen iz spletne aplikacije, mobilne aplikacije, zalednega strežnika, podatkovne baze in dodatnih storitev, ki bodo razvite pri posameznih projektnih predmetih.

Uporabnik bo lahko preko spletne ali mobilne aplikacije pregledoval razpoložljive knjige, si ogledal njihove podrobnosti, rezerviral knjigo ter preveril stanje svojih rezervacij.

Administrator bo imel možnost dodajanja, urejanja in odstranjevanja knjig ter upravljanja paketnikov. Sistem bo omogočal tudi varno prijavo uporabnikov in nadzor dostopa do posameznih funkcionalnosti.

Projekt bomo izvajali postopoma in ga razdelili na dve glavni fazi. Prva faza bo zajemala načrtovanje in pripravo razvojnih okolij. Druga faza bo zajemala implementacijo funkcionalnosti, integracijo posameznih delov sistema, testiranje in odpravljanje napak.

2 Predstavitev ideje in cilji projekta

Glavna ideja projekta BookBox je izdelava sistema za izposajo knjig preko pametnega paketnika. Uporabnik bo lahko knjigo rezerviral preko aplikacije, nato pa jo prevzel na izbranem paketniku. Takšen sistem bi omogočil prevzem knjig tudi izven odpiralnega časa knjižnice ali druge ustanove.

Sistem bo sestavljen iz naslednjih delov:

- spletne aplikacije za uporabnike in administratorje;
- mobilne aplikacije za uporabnike;
- zalednega strežnika z API-potmi;
- podatkovne baze;
- storitve za obdelavo slik v okviru predmeta Osnove računalniškega vida;
- Docker zabojnikov in skripte za zagon sistema;
- Git repozitorija in Jira projekta za organizacijo dela.

Glavni cilji projekta so:

1. omogočiti registracijo in prijavo uporabnikov;
 2. omogočiti pregled in iskanje knjig;
 3. omogočiti rezervacijo izbrane knjige;
 4. omogočiti pregled lastnih rezervacij;
 5. omogočiti upravljanje paketnikov;
 6. zagotoviti varno komunikacijo z zalednim sistemom;
 7. pripraviti sistem tako, da ga je mogoče vzpostaviti z eno skripto;
 8. dokumentirati namestitev in uporabo aplikacije.
-

3 Organizacija dela v skupini

Projekt bomo izvedli v mikroskupini treh članov:

- Anže Jeger,
- Lan Podlesnik,
- Nik Deželak.

Naloge bomo razdelili glede na posamezne dele sistema in projektne predmete. Pri večjih nalogah bo določen glavni odgovorni član, ostala člana pa bosta sodelovala pri testiranju, pregledu kode in integraciji.

Za načrtovanje in spremljanje dela bomo uporabljali orodje Jira. Vsaka večja funkcionalnost bo zapisana kot samostojna naloga oziroma *task*. Kompleksnejše funkcionalnosti bomo po potrebi razdelili na manjše podnaloge. Pri vsaki nalogi bomo določili odgovorno osebo, rok izvedbe in trenutno stanje.

Uporabljali bomo naslednja stanja nalog:

- **To Do** – naloga še ni bila začeta;
- **In Progress** – izvajanje naloge trenutno poteka;
- **Testing** – funkcionalnost se testira;
- **Done** – naloga je zaključena.

4 FAZA 1: VZPOSTAVITEV OKOLIJ IN PRIPRAVA PROJEKTA

4.1 Načrtovanje arhitekture sistema

Pred začetkom implementacije bomo določili osnovno arhitekturo sistema. Spletna in mobilna aplikacija bosta komunicirali z zalednim strežnikom preko API-klicev. Zaledni strežnik bo skrbel za poslovno logiko in komunikacijo s podatkovno bazo MongoDB.

Osnovni potek komunikacije bo naslednji:

1. uporabnik izvede dejanje v spletni ali mobilni aplikaciji;
2. aplikacija pošlje zahtevo na ustrezno API-pot;
3. zaledni strežnik preveri pravilnost zahteve in dovoljenja uporabnika;
4. strežnik po potrebi prebere ali spremeni podatke v podatkovni bazi;
5. odgovor se aplikaciji vrne v obliki JSON;
6. aplikacija uporabniku prikaže rezultat.

Predvideni termin izvedbe: **od 25. 4. 2026 do 5. 5. 2026.**

4.2 Vzpostavitev Git repozitorija

Za verzijsko vodenje kode bomo uporabljali Git in oddaljeni repozitorij GitHub. V repozitoriju bomo pripravili osnovno strukturo map, da bodo posamezni deli projekta med seboj ločeni in pregledni.

Predvidena struktura repozitorija:

```
PROJECT_PAMETNI_PAKETNIK/  
├── Internet_App/  
├── Phone_App/  
├── ORV/  
├── Sistemska_Admin/  
├── Dokumentacija/  
├── .gitignore  
└── README.md
```

Vsak član skupine bo spremembe razvijal na svoji veji. Ko bo posamezna funkcionalnost dokončana in preverjena, bomo ustvarili *pull request* ter spremembe združili v glavno vejo `main`.

Pri delu bomo uporabljali smiselna sporočila ob oddaji sprememb. Sporočila bodo vsebovala tudi oznako naloge iz Jire, na primer:

```
SCRUM-12 Dodana prijava uporabnika  
SCRUM-18 Dodan prikaz seznama knjig  
SCRUM-25 Dodana Docker konfiguracija za backend
```

V datoteko `.gitignore` bomo dodali datoteke in mape, ki se ne smejo shranjevati v repozitorij, na primer `node_modules`, `.env`, začasne datoteke, dnevnik in generirane datoteke.

Predvideni termin izvedbe: **do 7. 5. 2026.**

4.3 Vzpostavitev projekta v orodju Jira

V orodju Jira bomo ustvarili projekt, v katerem bomo evidentirali vse pomembne naloge. Naloge bomo razvrstili glede na posamezne predmete in dele sistema.

Predvidene skupine nalog:

- spletna aplikacija;
- mobilna aplikacija;
- računalniški vid;
- sistemska administracija;
- varnost;
- dokumentacija;
- testiranje in odpravljanje napak.

Vsaka naloga bo vsebovala kratek opis, odgovorno osebo, predvideni rok in stanje izvajanja. Napredek bomo preverjali najmanj enkrat tedensko. Ob zaključku posamezne naloge bomo preverili, ali je bila koda shranjena v Git repozitorij in ustrezno dokumentirana.

Predvideni termin izvedbe: **do 7. 5. 2026.**

4.4 Priprava okolja za spletno aplikacijo

Pri predmetu Razvoj aplikacij za internet bomo izdelali spletno aplikacijo s tehnologijami sklada MERN:

- MongoDB,
- Express,
- React,
- Node.js.

Najprej bomo vzpostavili zaledni del aplikacije. Ustvarili bomo Node.js projekt, namestili Express in pripravili osnovno strukturo map za modele, kontrolerje in API-poti.

Na zalednem delu bomo pripravili podatkovne modele:

- **User** – podatki o uporabniku;
- **Book** – podatki o knjigi;
- **PacketBox** – podatki o paketniku;
- **Reservation** – podatki o rezervaciji knjige;
- **Borrow** – podatki o izposoji;
- **RefreshToken** – token za shanjevanje uporabnika po vpisu.

Vzpostavili bomo tudi povezavo s podatkovno bazo MongoDB. Občutljive podatke, kot so povezovalni niz podatkovne baze in skrivnosti za seje uporabnikov, bomo hranili v datoteki `.env`, ki ne bo vključena v Git repozitorij.

Na čelnem delu bomo ustvarili React projekt in pripravili osnovne komponente, navigacijo ter začetno postavitev strani. Načrtovani bodo zasloni za prijavo, registracijo, seznam knjig, podrobnosti knjige, pregled rezervacij in uporabniški profil.

Predvideni termini izvedbe:

Aktivnost	Rok
Vzpostavitev backend projekta	10. 5. 2026
Priprava podatkovnih modelov	12. 5. 2026
Povezava z MongoDB	12. 5. 2026
Vzpostavitev React projekta	15. 5. 2026
Priprava osnovne navigacije	15. 5. 2026

4.5 Priprava okolja za mobilno aplikacijo

Za izdelavo mobilne aplikacije bomo uporabili Android Studio in Jetpack Compose. V prvi fazi bomo pripravili začetni projekt, določili osnovno strukturo kode in ustvarili osnovne zaslone.

Mobilna aplikacija bo vsebovala naslednje zaslone:

- zemljevid paketnikov;
- seznam knjig;
- pregled izposojenih oziroma rezerviranih knjig;
- profil uporabnika;
- podrobnosti izbrane knjige;
- zaslon za skeniranje QR kode;
- zaslon z izbiro kaj bo uporabnik naredil z knjigo.

Že v prvi fazi bomo načrtovali način povezovanja z zalednim API-jem. Poskrbeli bomo, da bodo komponente pripravljene tako, da bo pozneje mogoče enostavno priključiti dejanske podatke iz podatkovne baze.

Predvideni termin izvedbe: **od 12. 5. 2026 do 20. 5. 2026.**

4.6 Priprava okolja za računalniški vid

Pri predmetu Osnove računalniškega vida bomo pripravili ločeno okolje v programskem jeziku Python. Uporabili bomo knjižnice za obdelavo slik, predvsem OpenCV in NumPy.

Najprej bomo pripravili zbirko slik in določili strukturo map za učno in testno množico. Slike bomo pred obdelavo poenotili na enako velikost. Pri spreminjanju velikosti bomo ohranili razmerje stranic, manjkajoči prostor pa zapolnili s črnim robom.

V okviru tega dela projekta bomo preučili možnosti vključitve algoritmov računalniškega vida v končni sistem. Ena izmed možnosti je uporaba prepoznavne obraza kot dodatnega varnostnega mehanizma. Če neposredna vključitev ne bo tehnično izvedljiva v razpoložljivem času, bomo algoritem pripravili kot ločeno storitev oziroma prototip in jasno dokumentirali njegovo uporabo.

Predvideni termini izvedbe:

Aktivnost	Rok
Priprava Python okolja	20. 5. 2026
Namestitev knjižnic	20. 5. 2026
Priprava zbirke slik	24. 5. 2026
Osnovna obdelava slik	25. 5. 2026

4.7 Priprava infrastrukture in Docker okolja

Pri predmetu Sistemska administracija bomo razmislili, kako posamezne dele aplikacije zapakirati v Docker zabojnike. Cilj je omogočiti vzpostavitev celotnega sistema z enim ukazom oziroma eno skripto.

Predvideni Docker zabojniki:

- zabojnik za spletni frontend;
- zabojnik za backend;
- povezava z MongoDB;
- po potrebi zabojnik za dodatno storitev računalniškega vida.

Za upravljanje več zabojnikov bomo uporabili datoteko `docker-compose.yml`. S tem bo mogoče celoten sistem zagnati z ukazom:

```
docker compose up --build
```

Pripravili bomo tudi skripto za poenostavljen zagon sistema. Skripta bo preverila potrebne nastavitve in zagnala vse zabojnike.

Predvideni termini izvedbe:

Aktivnost	Rok
Načrt Docker arhitekture	30. 5. 2026
Dockerfile za backend	5. 6. 2026
Dockerfile za frontend	5. 6. 2026
Priprava <code>docker-compose.yml</code>	5. 6. 2026
Skripta za zagon sistema	5. 6. 2026

4.8 Načrt varnosti sistema

Varnost bomo upoštevali že v prvi fazi razvoja. Ker bo aplikacija obdelovala podatke o uporabnikih in rezervacijah, bomo omejili dostop do zaščitene funkcionalnosti.

Načrtovani varnostni ukrepi:

- varno shranjevanje gesel z uporabo zgoščevalne funkcije;
- uporaba knjižnice `bcrypt`;
- preverjanje uporabniških sej;
- zaščita API-poti, ki zahtevajo prijavo;
- preverjanje veljavnosti podatkov na zalednem delu;
- uporaba datoteke `.env` za občutljive nastavitve;
- omejen dostop do podatkovne baze;
- redno preverjanje, da skrivnosti niso objavljene v Git repozitoriju.

Pri podatkovni bazi bomo omejili dostop na zaupanja vredne uporabnike in omrežja. V produkcijskem okolju ne bomo uporabljali odprtega dostopa iz vseh naslovov IP, razen kadar bo to začasno potrebno med razvojem in testiranjem.

Predvideni termin izvedbe osnovnega varnostnega načrta: **do 25. 5. 2026.**

5 FAZA 2: IMPLEMENTACIJA, INTEGRACIJA IN TESTIRANJE

5.1 Implementacija spletne aplikacije

V drugi fazi bomo implementirali glavne funkcionalnosti spletne aplikacije. Na zalednem delu bomo dokončali kontrolerje in API-poti, na čelnem delu pa izdelali uporabniške zaslone in jih povezali z zaledjem.

Predvidene funkcionalnosti:

Funkcionalnost	Opis	Rok
Registracija uporabnika	Ustvarjanje novega uporabniškega računa	17. 5. 2026
Prijava in odjava	Upravljanje uporabniške seje	18. 5. 2026
Pregled knjig	Prikaz seznama knjig iz podatkovne baze	20. 5. 2026
Podrobnosti knjige	Prikaz avtorja, naslova, opisa in razpoložljivosti	21. 5. 2026
Rezervacija knjige	Ustvarjanje rezervacije izbrane knjige	23. 5. 2026
Pregled rezervacij	Pregled rezervacij prijavljenega uporabnika	24. 5. 2026
Upravljanje paketnikov	Dodajanje in urejanje paketnikov	25. 5. 2026
Uporabniški profil	Prikaz podatkov in možnost odjave	25. 5. 2026

Pri implementaciji bomo sproti preverjali pravilnost delovanja API-poti s programom Postman oziroma podobnim orodjem.

5.2 Implementacija mobilne aplikacije

Mobilna aplikacija bo omogočala pregled najpomembnejših funkcionalnosti na mobilni napravi. Implementirali bomo navigacijo med zaslone in prikaz podatkov.

Predvidene aktivnosti:

Aktivnost	Rok
Implementacija spodnje navigacije	22. 5. 2026
Zaslon z zemljevidom paketnikov	23. 5. 2026
Prikaz seznama knjig	24. 5. 2026

Aktivnost	Rok
Prikaz podrobnosti knjige	25. 5. 2026
Prikaz uporabnikovega profila	26. 5. 2026
Povezava z API-jem	28. 5. 2026
Testiranje na emulatorju	29. 5. 2026

Aplikacijo bomo najprej testirali na emulatorju, nato pa po možnosti tudi na dejanski mobilni napravi.

5.3 Implementacija algoritmov računalniškega vida

Pri predmetu Osnove računalniškega vida bomo razvili algoritem za obdelavo oziroma klasifikacijo slik. Pred uporabo algoritma bomo slike ustrezno predobdelali, jih poenotili na izbrano velikost in razdelili na učno ter testno množico.

Predvidene aktivnosti:

Aktivnost	Rok
Zbiranje in urejanje slik	20. 5. 2026
Predobdelava slik	23. 5. 2026
Implementacija algoritma	26. 5. 2026
Testiranje algoritma	28. 5. 2026
Analiza rezultatov	30. 5. 2026
Dokumentacija postopka	5. 6. 2026

Če bo algoritem neposredno vključen v končni produkt, bomo pripravili vmesnik za komunikacijo z ostalimi deli sistema. Če bo zaradi časovnih ali tehničnih omejitev deloval ločeno, bomo jasno opisali način zagona in možne nadaljnje korake za integracijo.

5.4 Kontejnerizacija in avtomatiziran zagon

Po dokončanju osnovnih funkcionalnosti bomo posamezne dele sistema zapakirali v Docker zabojnike. Posebno pozornost bomo namenili temu, da se odvisnosti ne bodo prenašale iz lokalnega računalnika v zabojnik. Namesto tega se bodo knjižnice namestile neposredno med gradnjo slike.

Preverili bomo:

- ali se backend pravilno zažene v zabojniku;
- ali se frontend poveže z ustreznim API-naslovom;
- ali se okoljske spremenljivke pravilno preberejo;
- ali sistem deluje tudi po ponovni gradnji slik;
- ali je mogoče sistem vzpostaviti z eno skripto;

- ali so mape `node_modules` izključene iz Docker konteksta.

Za zmanjšanje velikosti Docker konteksta bomo uporabili datoteko `.dockerignore`.

Predvideni termin izvedbe: **od 24. 5. 2026 do 30. 5. 2026.**

5.5 Integracija storitev

Po implementaciji posameznih delov bomo izvedli integracijo sistema. Cilj je preveriti, ali spletna aplikacija, mobilna aplikacija, backend, podatkovna baza in dodatne storitve pravilno sodelujejo.

Preverili bomo naslednje scenarije:

1. uporabnik se registrira;
2. uporabnik se prijavi;
3. aplikacija pridobi seznam knjig iz podatkovne baze;
4. uporabnik izbere knjigo;
5. uporabnik ustvari rezervacijo;
6. rezervacija se pravilno shrani v podatkovno bazo;
7. uporabnik vidi rezervacijo v svojem profilu;
8. administrator lahko upravlja podatke o knjigah in paketnikih;
9. sistem je mogoče zagnati z uporabo Docker skripte.

Predvideni termin izvedbe: **od 25. 5. 2026 do 1. 6. 2026.**

5.6 Testiranje in odpravljanje napak

Na koncu bomo izvedli testiranje celotnega sistema. Testiranje ne bo izvedeno samo ob koncu projekta, ampak bomo osnovne funkcionalnosti preverjali že sproti.

Izvedli bomo naslednje vrste testiranja:

Testiranje spletne aplikacije

Preverili bomo:

- registracijo z veljavnimi in neveljavnimi podatki;
- prijavo s pravilnim in napačnim geslom;
- odjavo uporabnika;
- prikaz knjig;
- ustvarjanje rezervacij;
- zaščito strani, ki zahtevajo prijavo;
- prikaz ustreznih obvestil ob napakah.

Testiranje mobilne aplikacije

Preverili bomo:

- navigacijo med zasloni;
- prikaz podatkov;
- delovanje na emulatorju;
- povezavo z API-jem;
- odziv aplikacije ob nedosegljivem strežniku.

Testiranje infrastrukture

Preverili bomo:

- gradnjo Docker slik;
- zagon sistema z eno skripto;
- komunikacijo med zabojniki;
- pravilno uporabo okoljskih spremenljivk;
- delovanje po ponovnem zagonu.

Testiranje varnosti

Preverili bomo:

- ali neprijavljen uporabnik ne more dostopati do zaščitene API-poti;
- ali se gesla ne shranjujejo v berljivi obliki;
- ali datoteka `.env` ni vključena v repozitorij;
- ali API zavrne nepravilne podatke;
- ali so dovoljenja uporabnikov ustrezno preverjena.

Predvideni termin izvedbe: **od 30. 5. 2026 do 2. 6. 2026.**

6 PREGLED TERMINSKEGA PLANA

Faza	Aktivnost	Predvideni termin	Odgovorni član
Faza 1	Načrt arhitekture sistema	7. 4.–10. 4. 2026	Celotna skupina
Faza 1	Git repozitorij in pravila dela	do 10. 4. 2026	Celotna skupina
Faza 1	Jira projekt in začetne naloge	do 10. 4. 2026	Celotna skupina
Faza 1	Priprava backend okolja	do 10. 5. 2026	Določeni član
Faza 1	Priprava frontend okolja	do 15. 5. 2026	Določeni član
Faza 1	Priprava mobilne aplikacije	do 20. 5. 2026	Določeni član
Faza 1	Priprava okolja za ORV	do 20. 5. 2026	Določeni član
Faza 1	Načrt Docker arhitekture	do 18. 5. 2026	Celotna skupina
Faza 2	Implementacija spletne aplikacije	16. 5.–25. 5. 2026	Določeni član
Faza 2	Implementacija mobilne aplikacije	22. 5.–29. 5. 2026	Določeni član

Faza	Aktivnost	Predvideni termin	Odgovorni član
Faza 2 Implementacija algoritma ORV		20. 5.–1. 6. 2026	Določeni član
Faza 2 Docker konfiguracija		24. 5.–30. 5. 2026	Celotna skupina
Faza 2 Integracija storitev		25. 5.–1. 6. 2026	Celotna skupina
Faza 2 Testiranje in odpravljanje napak		30. 5.–2. 6. 2026	Celotna skupina
Faza 2 Končna dokumentacija		do 3. 6. 2026	Celotna skupina

Pri stolpcu »Odgovorni član« bomo pred oddajo dokumenta dopisali konkretna imena glede na končno razdelitev dela v skupini.

7 PREDVIDENE NALOGE V ORODJU JIRA

Za lažji nadzor bomo v orodje Jira dodali najmanj naslednje naloge:

Oznaka naloge	Naslov naloge	Vrsta
SCRUM-1	Vzpostavitev Git repozitorija	Task
SCRUM-2	Priprava osnovne strukture projekta	Task
SCRUM-3	Vzpostavitev MongoDB podatkovne baze	Task
SCRUM-4	Implementacija podatkovnih modelov	Task
SCRUM-5	Implementacija registracije uporabnika	Task
SCRUM-6	Implementacija prijave in odjave	Task
SCRUM-7	Izdelava prikaza seznama knjig	Task
SCRUM-8	Izdelava rezervacije knjige	Task
SCRUM-9	Izdelava uporabniškega profila	Task
SCRUM-10	Priprava mobilne aplikacije	Task
SCRUM-11	Izdelava navigacije v mobilni aplikaciji	Task
SCRUM-12	Priprava ORV okolja	Task
SCRUM-13	Predobdelava zbirke slik	Task
SCRUM-14	Implementacija algoritma ORV	Task
SCRUM-15	Dockerfile za backend	Task
SCRUM-16	Dockerfile za frontend	Task
SCRUM-17	Priprava Docker Compose konfiguracije	Task
SCRUM-18	Priprava skripte za zagon	Task
SCRUM-19	Testiranje integracije	Task
SCRUM-20	Priprava dokumentacije	Task

Naloge bomo med razvojem po potrebi dopolnjevali in razdelili na manjše podnaloge.

8 TVEGANJA IN NAČRTOVANE REŠITVE

Pri razvoju lahko pride do težav, zato bomo vnaprej določili glavna tveganja in možne rešitve.

Tveganje	Možna posledica	Predvidena rešitev
Težave pri povezavi s podatkovno bazo,	Backend ne more dostopati do podatkov,	Preverjanje <code>.env</code> , omrežnega dostopa in uporabniških dovoljenj,
Sprememba lokalnega IP-naslova,	Mobilna aplikacija ne doseže API-ja,	Posodobitev naslova API-ja in dokumentiranje nastavitve,
Konflikti pri združevanju kode,	Izguba ali napačna združitev sprememb,	Delo na ločenih vejah in uporaba pull requestov,
Napačne odvisnosti v Docker zabojniku,	Aplikacija se v zabojniku ne zažene,	Namestitev knjižnic znotraj slike in uporaba <code>.dockerignore</code> ,
Premalo časa za neposredno integracijo ORV,	Algoritem ni vključen v končni produkt,	Priprava ločene storitve oziroma dokumentiranega prototipa,
Nepravilno shranjene skrivnosti,	Varnostno tveganje,	Uporaba <code>.env</code> in preverjanje datoteke <code>.gitignore</code> ,
Napake v API-poteh.	Nepravilno delovanje spletne ali mobilne aplikacije.	Sprotno testiranje s programom Postman.

9 DOKUMENTACIJA PROJEKTA

Ob zaključku bomo pripravili dokumentacijo za namestitev in uporabo sistema. Dokumentacija bo napisana tako, da jo lahko razume tudi uporabnik brez naprednega tehničnega znanja.

Dokumentacija bo vsebovala:

- opis sistema;
- zahteve za namestitev;
- navodila za zagon spletne aplikacije;
- navodila za zagon mobilne aplikacije;
- navodila za zagon ORV storitve oziroma algoritma;
- navodila za zagon sistema z Docker skripto;
- najmanj dva primera uporabe;
- opis morebitnih znanih omejitev.

Primer uporabe bo na primer rezervacija knjige: uporabnik se prijavi, pregleda seznam knjig, izbere knjigo, jo rezervira in preveri rezervacijo v svojem profilu.

10 ZAKLJUČEK

Terminski plan predstavlja osnovo za organizirano izvedbo projekta BookBox. Projekt je razdeljen na dve glavni fazi. V prvi fazi bomo pripravili okolja, repozitorij, podatkovno bazo, osnovno arhitekturo in varnostni načrt. V drugi fazi bomo implementirali funkcionalnosti, povezali posamezne dele sistema, pripravili Docker zabojnike in izvedli testiranje.

Z uporabo orodij GitHub in Jira bomo sproti spremljali napredek ter skrbeli za pregledno razdelitev dela. Takšen način dela bo omogočil lažje sodelovanje članov skupine in hitrejše odpravljanje napak.